

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №6: реализация межсервисного
взаимодействия посредством очередей сообщений

Выполнил:

Кузнецов Артур

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

Задача

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

В рамках лабораторной работы использовалось микросервисное приложение, состоящее из трёх основных сервисов: auth-service (аутентификация и пользователи), recipe-service (рецепты и связанные сущности) и social-service (лайки, комментарии, подписки, сохранённые рецепты). Каждый сервис работает в отдельном контейнере и имеет собственную базу данных PostgreSQL, что позволяет сервисам оставаться независимыми.

Для асинхронного обмена событиями был добавлен брокер сообщений RabbitMQ. Он поднимается отдельным контейнером в docker-compose.yml на стандартных портах:

- 5672 – AMQP для соединения приложений с брокером;
- 15672 – веб-интерфейс управления RabbitMQ.

Все сервисы и RabbitMQ подключены к общей docker сети app-network, поэтому внутри docker окружения брокер доступен по DNS-имени rabbitmq.

Для исключения дублирования кода логика работы с RabbitMQ была вынесена в общий модуль common-service, который подключается в остальные сервисы как локальная зависимость. Внутри него реализованы основные функции:

- подключение к RabbitMQ и создание AMQP-канала;
- отправка сообщений в очередь;
- получение сообщений из очереди с обработчиком колбэком.

В качестве канала обмена используется одна общая очередь, которую все сервисы объявляют одинаково при подключении. Сообщения передаются в JSON-формате: объект сериализуется в строку и кладётся в очередь.

Для устойчивости запуска реализованы повторные попытки соединения. Функция выполняет до 10 попыток подключения с задержкой 2 секунды между ними. Это важно для сценария запуска в `docker`, когда сервис может стартовать раньше, чем брокер полностью готов принимать подключения. Если подключиться не удалось даже после повторов, приложение завершает работу с ошибкой.

Отправка сообщений выполняется в тех местах, где происходят значимые изменения данных, например, при регистрации пользователя. После успешного выполнения операции сервис формирует сообщение вида:

- `event` – строковый тип события;
- полезные данные;
- `timestamp` – время формирования события.

Таким образом, каждый сервис публикует события своего домена, не вызывая напрямую другие сервисы.

При старте каждый микросервис выполняет одинаковую последовательность:

- 1) подключение к собственной базе данных PostgreSQL;
- 2) подключение к RabbitMQ;
- 3) запуск потребления очереди `recipe.events`.

Каждый сервис подписывается на общую очередь и получает все сообщения, появляющиеся в ней. В обработчике сообщений реализована базовая логика:

- вывод в лог типа события и содержимого сообщения;
- обработка отдельных событий как заготовка для реакций других сервисов.

После настройки `docker-compose.yml` и общего модуля RabbitMQ система была проверена в собранном окружении. При выполнении операций, например, регистрация пользователя, соответствующий сервис публикует событие в очередь. Остальные сервисы, подписанные на очередь, получают сообщение и отображают его в логах. Наличие повторных попыток подключения обеспечивает корректный старт сервисов при запуске всей системы контейнеров.

Вывод

В результате лабораторной работы был подключён и настроен брокер сообщений RabbitMQ в `docker` окружении микросервисного приложения. Для работы с очередями в общем модуле `common-service` реализованы функции подключения, отправки и получения сообщений, а также повторные попытки подключения при старте. Межсервисное взаимодействие реализовано асинхронно через публикацию и потребление доменных событий в общей очереди, что позволило сервисам обмениваться изменениями без прямых `http` запросов друг к другу и удовлетворило требованиям лабораторной работы.